
TRÍ TUỆ NHÂN TẠO

NumPy UltraQuick Tutorial

Dịch tiếng Việt từ Google ML Crash Course (Colab)

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Colab là gì?	4
2 Giới thiệu NumPy	4
3 Nhập (import) NumPy	4
4 Khởi tạo array với giá trị cụ thể	4
5 Khởi tạo array theo dãy số	5
6 Khởi tạo array ngẫu nhiên	5
7 Phép toán trên NumPy array và broadcasting	6
8 Bài tập 1: Tạo dữ liệu tuyến tính	6
9 Bài tập 2: Thêm nhiễu (noise) vào dataset	7

Lời nói đầu

Nguồn

Tài liệu dịch từ notebook *NumPy UltraQuick Tutorial* của Google Machine Learning Crash Course: https://colab.research.google.com/github/google/eng-edu/blob/main/ml/cc/exercises/numpy_ultraquick_tutorial.ipynb

Mục tiêu

Đây không phải tutorial NumPy đầy đủ. Mục tiêu là cung cấp **vừa đủ** NumPy để làm các bài tập Colab trong ML Crash Course.

Đặng Tấn Quý

1 Colab là gì?

Colaboratory (Colab)

Machine Learning Crash Course dùng **Colaboratory (Colab)** cho tất cả bài tập lập trình. Colab là một hiện thực của Google dựa trên **Jupyter Notebook**: <https://jupyter.org/>.

Để tìm hiểu cách dùng Colab, xem *Welcome to Colaboratory*: <https://research.google.com/colaboratory>.

2 Giới thiệu NumPy

Vì sao cần NumPy?

NumPy là thư viện Python để tạo và thao tác **ma trận** (cấu trúc dữ liệu trung tâm của nhiều thuật toán ML). Ma trận là đối tượng toán học lưu giá trị theo **hàng** và **cột**.

Cách gọi “ma trận” trong các thư viện

Trong Python, “ma trận” thường được biểu diễn bằng **list**. Trong NumPy, “ma trận” thường được gọi là **array**. Trong TensorFlow, “ma trận/mảng đa chiều” thường được gọi là **tensor**.

3 Nhập (import) NumPy

Import module NumPy

Chạy cell sau để import NumPy.

```
import numpy as np
```

4 Khởi tạo array với giá trị cụ thể

Tạo array 1 chiều bằng np.array

Gọi `np.array` để tạo một NumPy array với các giá trị bạn chọn. Ví dụ sau tạo một array 8 phần tử.

```
one_dimensional_array = np.array([1.2, 2.4, 3.5, 4.7, 6.1, 7.2, 8.3, 9.5])  
print(one_dimensional_array)
```

Tạo array 2 chiều (ma trận)

Để tạo array 2 chiều, bạn đặt thêm một lớp ngoặc vuông. Ví dụ sau tạo ma trận 3×2 .

```
two_dimensional_array = np.array([[6, 5], [11, 7], [4, 8]])
print(two_dimensional_array)
```

Array toàn số 0 và toàn số 1

Để tạo array toàn số 0, dùng `np.zeros`. Để tạo array toàn số 1, dùng `np.ones`.

5 Khởi tạo array theo dãy số

Tạo dãy số bằng `np.arange`

Bạn có thể tạo array từ một dãy số liên tiếp.

```
sequence_of_integers = np.arange(5, 12)
print(sequence_of_integers)
```

Cận dưới và cận trên trong `np.arange`

`np.arange(5, 12)` tạo dãy có 5 nhưng **không** gồm 12.

6 Khởi tạo array ngẫu nhiên

Số nguyên ngẫu nhiên bằng `np.random.randint`

`np.random.randint` sinh số nguyên ngẫu nhiên trong khoảng `[low, high)`. Ví dụ sau tạo array 6 phần tử trong khoảng 50 đến 100.

```
random_integers_between_50_and_100 = np.random.randint(low=50, high=101, size=(6,))
print(random_integers_between_50_and_100)
```

Giá trị lớn nhất sinh ra

Số lớn nhất mà `np.random.randint` có thể sinh ra luôn **nhỏ hơn** đối số `high`. Vì vậy để có thể sinh 100, ta đặt `high=101`.

Số thực ngẫu nhiên bằng `np.random.random`

`np.random.random` sinh số thực ngẫu nhiên trong `[0.0, 1.0)`.

```
random_floats_between_0_and_1 = np.random.random((6,))
print(random_floats_between_0_and_1)
```

7 Phép toán trên NumPy array và broadcasting

Vì sao cần broadcasting?

Trong đại số tuyến tính, phép cộng/trừ hai array thường yêu cầu cùng kích thước. Phép nhân cũng có quy tắc tương thích kích thước chặt chẽ. NumPy hỗ trợ một cơ chế gọi là **broadcasting** để “mở rộng ảo” toán hạng nhỏ hơn cho phù hợp kích thước.

Cộng một hằng số (broadcasting)

Ví dụ sau cộng 2.0 vào mọi phần tử của array.

```
random_floats_between_2_and_3 = random_floats_between_0_and_1 + 2.0
print(random_floats_between_2_and_3)
```

Nhân với một hằng số (broadcasting)

Ví dụ sau nhân mọi phần tử với 3.

```
random_integers_between_150_and_300 = random_integers_between_50_and_100 * 3
print(random_integers_between_150_and_300)
```

8 Bài tập 1: Tạo dữ liệu tuyến tính

Yêu cầu bài tập

Tạo dataset đơn giản gồm một đặc trưng (feature) và một nhãn (label):

1. Gán dãy số nguyên từ 6 đến 20 (bao gồm cả 6 và 20) vào NumPy array tên **feature**.
2. Gán 15 giá trị vào NumPy array tên **label** sao cho:

$$\text{label} = 3 \cdot \text{feature} + 4.$$

Ví dụ phần tử đầu tiên của **label** là:

$$3 \cdot 6 + 4 = 22.$$

```
feature = ? # write your code here
print(feature)
label = ? # write your code here
print(label)
```

Một lời giải tham khảo

```
feature = np.arange(6, 21)
print(feature)
label = (feature * 3) + 4
print(label)
```

9 Bài tập 2: Thêm nhiễu (noise) vào dataset

Yêu cầu bài tập

Để dữ liệu thực tế hơn, hãy thêm một chút nhiễu ngẫu nhiên vào từng phần tử của `label`:

- Mỗi phần tử của `label` được cộng với một số thực ngẫu nhiên **khác nhau** trong đoạn $[-2, 2]$.
- Không dựa vào broadcasting. Hãy tạo một array `noise` có cùng kích thước với `label`.

```
noise = ?    # write your code here
print(noise)
label = ?    # write your code here
print(label)
```

Một lời giải tham khảo

```
noise = (np.random.random([15]) * 4) - 2
print(noise)
label = label + noise
print(label)
```